

Project Syllo: Educational AI SaaS

The Definitive 2026 Engineering & Architecture Blueprint

Version: 2.0 (Production Candidate)

Scope: Full-Stack Specifications, ML Pipelines, Database Schemas, API Specs, and DevOps

Table of Contents

1. Executive Architecture Overview
2. The Frontend Layer: Next.js & React Ecosystem
3. The Backend Gateway: Next.js API & Auth
4. The Database Layer: MongoDB Atlas & Vector Search
5. Database Schemas (Mongoose Definitions)
6. The AI Microservice: Python FastAPI Architecture
7. Asynchronous Task Management: Celery & Redis
8. Machine Learning Pipeline: Document Ingestion & RAG
9. LLM Integration: Google Gemini API & Structured Outputs
10. Prompt Engineering Matrix
11. Multi-Modal Extensions (Image Gen & Knowledge Graphs)
12. Google Workspace Integration (Drive, Classroom, Forms)
13. Complete API Endpoint Specification
14. DevOps, Docker, and Zero-Cost Deployment Blueprint
15. Security, Rate Limiting, and Optimization

1. Executive Architecture Overview

Project Sylo is a highly scalable Educational Technology (EdTech) SaaS designed to ingest complex academic documents (PDFs, PPTXs, TXTs) and autonomously generate interactive learning interfaces. The core features include dynamic multiple-choice quizzes, spaced-repetition flashcards, visual concept maps (Knowledge Graphs), and automated Google Form deployments.

1.1 Microservices Architecture Rationale

A monolithic Node.js application is fundamentally incompatible with heavy Machine Learning workloads. Node.js is single-threaded; parsing a 200-page PDF and calculating 10,000 vector embeddings would block the event loop, causing the entire website to freeze for all concurrent users. Therefore, this project implements a strict decoupled microservices architecture:

- **Client / Web Server:** A Next.js application handling UI, routing, database CRUD operations, and user sessions.
- **Message Broker:** A Redis instance managing an asynchronous job queue.
- **AI Worker:** A Python-based FastAPI server that strictly handles background ML processing (LangChain, PyPDF, Gemini API).
- **Unified Data Store:** MongoDB Atlas, functioning simultaneously as a NoSQL document store and a High-Dimensional Vector Search engine.

System Flow Diagram (Text Representation)

```
[User Browser] -> HTTP POST /upload -> [Next.js Gateway]
                                     |
                                     v (Writes Document metadata to DB)
                               [MongoDB Atlas]
                                     ^
```

[Next.js Gateway] -> HTTP POST /enqueue -> [FastAPI Server] -> Adds to [Redis Queue]

(Pulls Job)

|
[Celery Worker]

vectors

|
[PyPDF2] extracts text
[LangChain] chunks text
[Embedding API] creates

MongoDB)

|
(Saves Vectors back to

2. The Frontend Layer: Next.js & React Ecosystem

The frontend is built on Next.js 14+ utilizing the App Router paradigm. This allows for React Server Components (RSC) to handle initial data fetching securely on the server before sending HTML to the client, greatly improving performance and SEO.

2.1 UI/UX Design System

The design language strictly adheres to Neo-Glassmorphism and Bento Grid layouts. This ensures the dashboard handles dense data gracefully.

- **Tailwind CSS:** Core utility-first styling engine.
- **Shadcn UI:** Unstyled, accessible React components configured for a minimal aesthetic.
- **Framer Motion:** For orchestrating complex layout transitions, especially when polling for background ML tasks to finish.
- **React Flow / Cytoscape:** Libraries utilized to render the JSON outputs of the AI into interactive, draggable Knowledge Graphs.

2.2 State Management & Polling

Because the AI generation takes up to 45 seconds, the frontend must implement optimistic UI updates using `@tanstack/react-query`. When a generation request is sent, the UI transitions to a "Processing Environment" that polls the backend.

```
// Example Polling Implementation in React Component
import { useQuery } from '@tanstack/react-query';

function GenerationStatus({ jobId }) {
  const { data, isLoading } = useQuery({
    queryKey: ['jobStatus', jobId],
    queryFn: () => fetch(`/api/jobs/${jobId}`).then(res => res.json()),
  });
```

```

    refetchInterval: (data) => (data?.status === 'COMPLETED' ? false : 2000), //
    Poll every 2s
  });

  if (isLoading || data?.status === 'PROCESSING') return <AnimatedLoader
    phase={data?.phase} />;
  return <QuizDashboard data={data.result} />;
}

```

3. The Backend Gateway: Next.js API & Auth

The Next.js backend serves as the orchestration layer. It never performs heavy compute; it validates requests, interacts with MongoDB, triggers the Python worker, and handles user authentication.

3.1 Authentication & Scopes (NextAuth.js)

Authentication is handled via OAuth 2.0. To enable seamless Google Drive and Classroom ingestion, the application must request elevated scopes upon user sign-in.

```

// pages/api/auth/[...nextauth].js
import GoogleProvider from "next-auth/providers/google";

export const authOptions = {
  providers: [
    GoogleProvider({
      clientId: process.env.GOOGLE_ID,
      clientSecret: process.env.GOOGLE_SECRET,
      authorization: {
        params: {
          prompt: "consent",
          access_type: "offline", // Crucial for obtaining refresh tokens
          response_type: "code",
          scope: "openid email profile https://www.googleapis.com/auth/
drive.readonly https://www.googleapis.com/auth/classroom.courses.readonly https://

```

```
www.googleapis.com/auth/forms.body"
    }
  }
})
],
callbacks: {
  async jwt({ token, account }) {
    if (account) {
      token.accessToken = account.access_token; // Pass token for API calls
    }
    return token;
  }
}
}
```

4. The Database Layer: MongoDB Atlas & Vector Search

MongoDB Atlas is utilized as a unified data layer. This eliminates the latency and synchronization issues of maintaining a separate relational database (like Postgres) and a vector database (like Pinecone).

4.1 Atlas Vector Search Configuration

To enable semantic search over textbook chunks, a specific index must be created on the `DocumentChunks` collection. The embedding model outputs vectors of 768 dimensions.

```
// MongoDB Atlas Search Index Definition
{
  "mappings": {
    "dynamic": true,
    "fields": {
      "embedding": {
        "dimensions": 768,
        "similarity": "cosine",
        "type": "knnVector"
      }
    }
  }
}
```

5. Database Schemas (Mongoose Definitions)

The application requires a robust relational structure mapped into MongoDB's document model.

5.1 The User Schema

```
const UserSchema = new mongoose.Schema({
  email: { type: String, required: true, unique: true },
  name: String,
  oauthProvider: String,
  subscriptionTier: { type: String, enum: ['FREE', 'PRO'], default: 'FREE' },
  usageMetrics: {
    documentsProcessed: { type: Number, default: 0 },
    tokensConsumed: { type: Number, default: 0 }
  },
  createdAt: { type: Date, default: Date.now }
});
```

5.2 The Document & Chunk Schemas

A parent document represents the uploaded file. The chunks represent the semantic paragraphs stored with vector embeddings for the RAG pipeline.

```
const DocumentSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  title: String,
  fileUrl: String, // Link to S3 or Google Cloud Storage
  pageCount: Number,
  processingStatus: { type: String, enum: ['PENDING', 'CHUNKING', 'EMBEDDING', 'READY' ] },
});

const DocumentChunkSchema = new mongoose.Schema({
  documentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Document' },
  pageNumber: Number,
  text: String, // The actual text paragraph
  embedding: [Number], // The 768-dimensional float array
});
```

5.3 The Generated Assessment Schema

```
const QuizSchema = new mongoose.Schema({
  documentId: { type: mongoose.Schema.Types.ObjectId, ref: 'Document' },
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  parameters: {
    difficulty: String,
    questionCount: Number
  },
  questions: [{
    questionText: String,
    options: [String],
    correctAnswerIndex: Number,
    explanation: String
  }],
  googleFormUrl: String, // Populated post-export
  createdAt: { type: Date, default: Date.now }
});
```

6. The AI Microservice: Python FastAPI Architecture

The FastAPI server acts as the dedicated ML pipeline. It is entirely stateless, reading data from cloud storage, processing it, writing to MongoDB, and reporting status back to Redis.

6.1 Required Dependencies (requirements.txt)

```
fastapi==0.103.1
uvicorn==0.23.2
celery==5.3.4
redis==5.0.0
pymongo==4.5.0
langchain==0.0.300
PyPDF2==3.0.1
google-generativeai==0.3.1
sentence-transformers==2.2.2 # For local free-tier embeddings
```

6.2 FastAPI Endpoint Configuration

```
from fastapi import FastAPI, BackgroundTasks
from pydantic import BaseModel
from celery_worker import process_document_task, generate_quiz_task

app = FastAPI(title="Syllo AI Engine")

class DocumentIngestRequest(BaseModel):
    document_id: str
    file_url: str

@app.post("/v1/process")
```

```
async def process_document(request: DocumentIngestRequest):  
    # Offload to Celery worker immediately  
    task = process_document_task.delay(request.document_id, request.file_url)  
    return {"status": "queued", "task_id": task.id}
```

7. Asynchronous Task Management: Celery & Redis

When FastAPI receives the trigger, it pushes the workload to a Celery worker. Redis acts as the message broker passing tasks between FastAPI and Celery.

```
# celery_worker.py  
from celery import Celery  
from ml_pipeline import download_and_extract_pdf, generate_embeddings  
  
celery_app = Celery(  
    "ai_tasks",  
    broker="redis://red-xxxxx.upstash.io:6379",  
    backend="redis://red-xxxxx.upstash.io:6379"  
)  
  
@celery_app.task(bind=True)  
def process_document_task(self, document_id, file_url):  
    try:  
        # Step 1: Download & Extract  
        raw_text = download_and_extract_pdf(file_url)  
        self.update_state(state='PROGRESS', meta={'phase': 'CHUNKING'})  
  
        # Step 2: Generate Vector Embeddings and Save to MongoDB  
        generate_embeddings(document_id, raw_text)  
  
        return {"status": "SUCCESS", "document_id": document_id}  
    except Exception as e:  
        self.update_state(state='FAILURE', meta={'error': str(e)})
```

raise

8. Machine Learning Pipeline: Document Ingestion & RAG

Retrieval-Augmented Generation (RAG) is the core algorithm that makes this application functional without needing to train custom models. It bridges the gap between massive textbooks and the LLM's context window limits.

8.1 Recursive Character Chunking

To ensure vectors encapsulate complete thoughts, text is not split arbitrarily. LangChain's `RecursiveCharacterTextSplitter` breaks text primarily by paragraphs (), then by sentences (), and finally by words.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    length_function=len,
    separators=["

", "
", ". ", " ", ""]
)

chunks = text_splitter.split_text(raw_document_text)
```

Optimization Note: The 150-character overlap ensures that if a concept spans across two chunks, the AI retains the context in both vector representations.

8.2 Vector Search Execution

When a quiz request is made (e.g., "Quiz on Chapter 3: Thermodynamics"), the backend converts the query to a vector and uses a MongoDB `$vectorSearch` pipeline stage to pull the top 5 relevant chunks.

```
# MongoDB Aggregation Pipeline for Semantic Search
pipeline = [
    {
        "$vectorSearch": {
            "index": "vector_index",
            "path": "embedding",
            "queryVector": query_embedding_array,
            "numCandidates": 50,
            "limit": 5,
            "filter": {"documentId": {"$eq": target_document_id}}
        }
    },
    { "$project": { "_id": 0, "text": 1, "score": {"$meta": "vectorSearchScore"} } }
]
```

9. LLM Integration: Google Gemini API & Structured Outputs

The retrieved chunks are injected into a prompt for Google's Gemini 2.5 Flash model. The most critical engineering task here is forcing the LLM to return valid JSON, otherwise, the React frontend will crash.

```
import google.generativeai as genai

# Configure API
genai.configure(api_key="YOUR_GEMINI_API_KEY")

# Define the exact JSON structure we expect
```

```
generation_config = genai.types.GenerationConfig(
    temperature=0.2, # Low temperature for factual accuracy
    response_mime_type="application/json",
    response_schema={
        "type": "object",
        "properties": {
            "quiz_title": {"type": "string"},
            "questions": {
                "type": "array",
                "items": {
                    "type": "object",
                    "properties": {
                        "question": {"type": "string"},
                        "options": {
                            "type": "array",
                            "items": {"type": "string"}
                        },
                        "correct_answer": {"type": "string"},
                        "explanation": {"type": "string"}
                    },
                    "required": ["question", "options", "correct_answer",
                                "explanation"]
                }
            },
            "required": ["quiz_title", "questions"]
        }
    }
)
```

10. Prompt Engineering Matrix

The system utilizes distinct instructional templates depending on the module (Quizzes, Flashcards, or Concept Graphs). These templates combine the User Parameters and the RAG Context.

Base Quiz Generation Prompt

```
You are an expert university professor in the field. Your task is to generate a highly accurate, academic-level assessment based ONLY on the provided context material. PARAMETERS: - Difficulty: {difficulty} - Total Questions: {question_count} - Topic Focus: {topic} CONTEXT: {rag_chunks_concatenated} INSTRUCTIONS: 1. Formulate questions that require critical thinking, not just verbatim recall. 2. Ensure the correct answer is unambiguously supported by the context. 3. Provide three plausible but incorrect distractors for options. 4. Provide a 1-2 sentence explanation detailing why the answer is correct based on the text. 5. You MUST format your response according to the JSON schema provided.
```

11. Multi-Modal Extensions

Beyond text-based quizzes, the platform provides visual learning aids.

11.1 Knowledge Graph Generation (Entity Extraction)

Instead of generating Q&A, the prompt instructs the LLM to act as a Named Entity Recognition (NER) system, extracting relationships.

```
# Expected output schema for Graph Generation
{
```

```
"nodes": [{"id": "Neural Networks", "group": "Algorithm"}],  
"links": [{"source": "Machine Learning", "target": "Neural Networks", "label":  
"includes"}]  
}
```

This JSON payload is piped directly into the `react-flow-renderer` library on the frontend to render interactive, draggable web mind-maps.

11.2 Image Generation Integration

For complex mechanical or physical concepts, the system extracts a visual description utilizing Gemini, then pings the free Hugging Face Inference API to generate a conceptual diagram.

```
# Python requests to Hugging Face FLUX/Stable Diffusion  
import requests  
  
API_URL = "https://api-inference.huggingface.co/models/black-forest-labs/FLUX.1-  
schnell"  
headers = {"Authorization": f"Bearer {HF_TOKEN}"}  
  
def generate_concept_image(prompt):  
    payload = {"inputs": prompt}  
    response = requests.post(API_URL, headers=headers, json=payload)  
    return response.content # Returns image bytes
```

12. Google Workspace Integration

The defining enterprise feature of this application is its ability to not just generate quizzes, but autonomously deploy them to the Google Workspace ecosystem as live, distributable Google Forms.

12.1 Programmatic Google Forms Export

Once the React dashboard receives the generated JSON quiz, the user clicks "Export to Google Forms". The Next.js backend uses the user's stored OAuth token to execute the following logic via the `googleapis` npm package.

```
// Next.js API Route for Google Forms Export
import { google } from 'googleapis';

export async function POST(req, res) {
  const { generatedQuiz, accessToken } = req.body;

  const oauth2Client = new google.auth.OAuth2();
  oauth2Client.setCredentials({ access_token: accessToken });
  const forms = google.forms({ version: 'v1', auth: oauth2Client });

  // Step 1: Create an empty form
  const createRes = await forms.forms.create({
    requestBody: { info: { title: generatedQuiz.quiz_title } }
  });
  const formId = createRes.data.formId;

  // Step 2: Build the batch update request
  const requests = generatedQuiz.questions.map((q, index) => ({
    createItem: {
      item: {
        title: q.question,
        questionItem: {
          question: {
            required: true,
```

```

        choiceQuestion: {
          type: 'RADIO',
          options: q.options.map(opt => ({ value: opt }))
        }
      },
      location: { index }
    }
  }
});

// Step 3: Mutate the form with the new questions
await forms.batchUpdate({
  formId,
  requestBody: { requests }
});

return res.status(200).json({ formUrl: createRes.data.responderUri });
}

```

13. Complete API Endpoint Specification

A summary of the primary RESTful interfaces facilitating system communication.

Route	Service	Method	Description & Payload
<code>/api/upload</code>	Next.js	POST	Receives multipart form data. Uploads file to object storage. Returns file URI.
<code>/api/jobs/ trigger</code>	Next.js	POST	Pings Python backend with File URI to begin processing. Returns Task ID.
<code>/api/jobs/ {id}</code>	Next.js	GET	Checks MongoDB for Celery task status. Used by frontend polling.

<code>/v1/ml/ process</code>	FastAPI	POST	Internal route. Triggers Celery worker for chunking and embedding.
<code>/v1/ml/ generate</code>	FastAPI	POST	Internal route. Executes RAG search, calls Gemini LLM, returns structured JSON.

14. DevOps, Docker, and Zero-Cost Deployment Blueprint

The entire production stack is engineered to run on robust free tiers suitable for student developers, utilizing containerization for reliability.

14.1 The Python Microservice Dockerfile (Hugging Face Spaces)

Hugging Face Spaces provides free hosting for Python applications. We package the FastAPI and Celery worker into a single Docker container using a supervisor to run both processes simultaneously.

```
FROM python:3.10-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# Expose port for FastAPI
EXPOSE 7860

# CMD config to run FastAPI (via Uvicorn)
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "7860"]
```

14.2 Frontend Deployment (Vercel)

The Next.js application is deployed to Vercel. Continuous Integration (CI) is natively supported by linking the GitHub repository. Environment variables required in the Vercel dashboard include:

- `MONGODB_URI` : Connection string to Atlas Cluster.
- `NEXTAUTH_SECRET` : Secure random string for JWT encryption.
- `GOOGLE_CLIENT_ID` & `GOOGLE_CLIENT_SECRET` : From GCP console.
- `FASTAPI_SERVICE_URL` : The public URL of the deployed Hugging Face space.

15. Security, Rate Limiting, and Optimization

Even on free tiers, security and cost-control are paramount to prevent abuse.

15.1 Route Protection & JWTs

All Next.js API routes are wrapped in NextAuth session checks. If a request lacks a valid JWT cookie, it is rejected with a `401 Unauthorized` status before any database action occurs.

15.2 Rate Limiting (Upstash Redis)

To prevent users from exhausting the Gemini API limits or crashing the Python worker, API routes implement a sliding-window rate limiter using Redis.

```
import { Ratelimit } from "@upstash/ratelimit";
import { Redis } from "@upstash/redis";

const ratelimit = new Ratelimit({
  redis: Redis.fromEnv(),
  limiter: Ratelimit.slidingWindow(5, "1 d"), // 5 generations per user per day
  analytics: true,
});
```

```
// Inside Next.js API Route
const { success } = await ratelimit.limit(userId);
if (!success) return new Response("Rate limit exceeded", { status: 429 });
```

15.3 Database Indexing

Beyond the Vector Search index, standard B-Tree indexes are applied to the `userId` field in all collections (Documents, Quizzes) to ensure $O(1)$ lookup times when loading user dashboards.

End of Architectural Blueprint

Generated Programmatically via AI Orchestration Pipeline